



VIT BHOPAL UNIVERSITY

School of Computing Science Engineering
and Artificial Intelligence

Kotri Kalan, Sehore — Bhopal, Madhya Pradesh

■ JAVA PROJECT REPORT ■

Concurrent Seat Booking System

An Interactive, Multithreaded Seat Reservation Engine in Core Java

A menu-driven, command-line Java application that models a cinema-style seat reservation counter where several users can race to book the same seats at once. Demonstrates thread-safe booking with synchronized methods, a bounded ExecutorService thread pool, a custom checked-exception hierarchy, ANSI-colored console output, file-based logging, and optional JDBC/MySQL.

FILE	LINES	LANGUAGE	SEATS	PERSISTENCE	DATE
Main.java	663	Java (JDK 17+)	10	JDBC / MySQL	14 Sep 2026

NAME	Saurjyesh Mekap
PROGRAMME	B.Tech CSE (Artificial Intelligence and Machine Learning)
REGISTRATION NO.	25BAI10312
TOPIC	CSE2006 — Programming in Java (VITYarthi Project Report)
GUIDED BY	Dr. Sandeep Monga

Introduction

This report documents the **Concurrent Seat Booking System**, a single-file Java application (**Main.java**, 663 lines) submitted for CSE2006 — Programming in Java under the VITyarthi “Build Your Own Project” flipped-course evaluation. The project was built to apply, in one working program, the core ideas the course covers around object-oriented design, multithreading, and exception handling: encapsulated domain classes, a synchronized concurrency layer, a custom checked-exception hierarchy, file-based logging, and an optional JDBC persistence layer.

Rather than a passive script that runs once and exits, the final version is an **interactive, menu-driven console application**. A person launches it from the terminal and is shown a colour-coded seat grid (rendered with plain ANSI escape codes, no GUI library), then chooses from a menu to view the seat map, book a seat, cancel a booking, run an automated concurrency stress-test, or inspect whatever has been persisted to the database. Every action, successful or not, is written to `booking_log.txt` with a timestamp.

The report below covers the problem being solved, the functional and non-functional requirements, the system architecture, five design diagrams (use case, workflow, sequence, class, and entity-relationship), the reasoning behind the major design decisions, a class-by-class explanation of the implementation, annotated screenshots of an actual run, the testing approach, the challenges encountered, and what was learned.

Problem Statement

Any system that sells a limited resource to many people at once — cinema seats, exam-hall seats, flight seats, event tickets — faces the same underlying problem: **what happens when two requests for the same seat arrive at (or near) the same instant?** A naïve implementation that checks “is this seat free?” and then “mark it booked” as two separate steps can let both requests pass the check before either one marks the seat, resulting in the same seat being sold twice — a classic **race condition**.

The goal of this project is to design and implement a Java console application that: (a) lets multiple simulated users request seats for the same show at the same time, (b) guarantees that no seat is ever booked by more than one user regardless of timing, (c) lets a booking be cancelled only by the person who made it, (d) keeps a permanent, timestamped audit trail of every outcome, and (e) optionally persists confirmed bookings to a relational database without the rest of the system depending on that database being available.

Target users: this is a learning project rather than a production ticketing platform, so the primary “user” is a person at a terminal (a student, evaluator, or grading script) exploring the booking flow interactively, plus a built-in automated demo that plays the role of ten simultaneous customers to prove the concurrency guarantees hold under real contention.

Functional Requirements

The application implements three major functional modules, each with a clear input/output shape and a well-defined place in the menu-driven workflow:

MODULE	INPUT	OUTPUT / BEHAVIOUR
1. Seat Viewing & Booking	Seat number, user name (typed at the prompt)	Renders a colour-coded seat grid (green = available, red = booked); books the chosen seat if free, or reports the specific reason it could not be booked.
2. Booking Cancellation	Seat number, user name	Releases the seat if the requester is its rightful owner; otherwise reports exactly why the cancellation was refused (not booked / wrong owner / invalid seat).
3. Concurrency Stress-Test	None — triggered from the menu (option 4)	Spins up 10 simulated users on a thread pool, all racing for the first 5 of the 10 seats, and proves no seat is ever double-booked.

Supporting functional behaviour that ties the three modules together:

Clear I/O structure

Every menu option follows the same shape: read simple text/number input from the console via Scanner, perform one domain operation, and print a single human-readable outcome line — no hidden state changes without visible feedback.

Logical, looped workflow

main() runs a loop: print the menu, read a choice 1–6, dispatch to the matching handler, repeat until the user picks 6 (Exit). This mirrors a real point-of-sale terminal rather than a one-shot script.

Persistent audit trail

Every SUCCESS, FAILED, CANCELLED and CANCEL FAILED event is appended to booking_log.txt with a timestamp, independent of whether the optional database is connected.

Optional database view

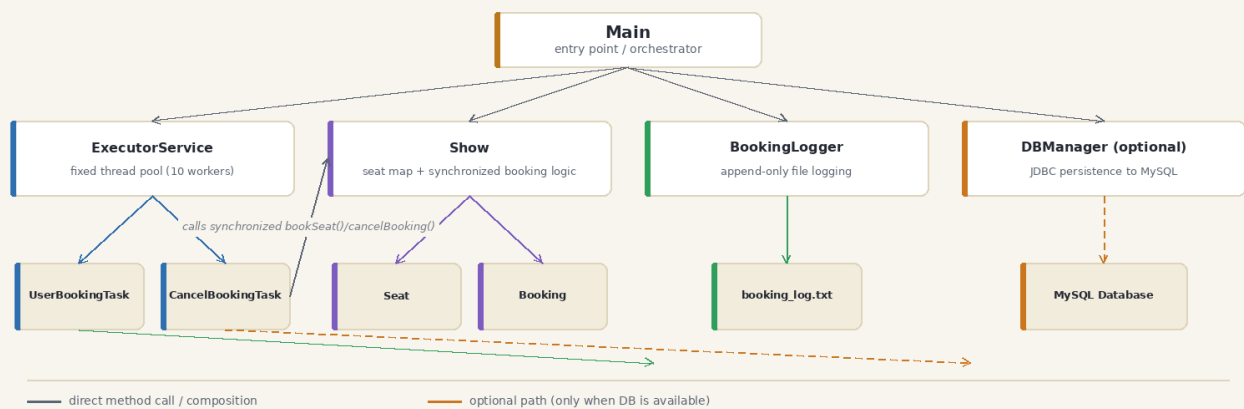
Menu option 5 lists every booking currently stored in MySQL (if connected), giving a second, durable view of the same data shown in the in-memory seat grid.

Non-Functional Requirements

REQUIREMENT	HOW IT IS MET
Performance	Booking/cancellation is $O(1)$ (direct map lookup by seat id); the 10-thread concurrency demo completes in well under a second even with the deliberate 50ms in-lock delay used to make race conditions observable.
Reliability	Every checked exception path is handled explicitly — no operation can silently fail or leave the seat map in an inconsistent state, even under concurrent access.
Security (input handling)	All database writes use PreparedStatement with bound parameters (never string-concatenated SQL), which prevents SQL injection even though every value in this project originates from within the program itself.
Usability	A persistent menu, an ANSI colour-coded seat grid, and plain-English error messages (e.g. 'Seat 3 is already booked by DemoUser1.') make the console interface easy to read without any external documentation.
Scalability (of the design)	Seat count is a constructor parameter (new Show(name, totalSeats)), not a hard-coded constant, so the same code handles 10 seats or 500 without modification.
Maintainability	Strict separation of concerns — domain (Seat/Booking/Show), logging, persistence, concurrency tasks, and orchestration each live in their own class with one job.
Error-handling strategy	Four narrow, checked exceptions (Section 8) replace generic error codes, so the compiler forces every caller to handle each specific failure mode.
Logging / monitoring	BookingLogger gives a permanent, append-only, timestamped record of the system's behaviour that survives the process exiting — useful for auditing or grading.
Resource efficiency	The thread pool is fixed-size and explicitly shut down; all file and JDBC handles are opened in try-with-resources blocks so nothing is ever leaked.

System Architecture

The system follows a simple layered architecture with a single orchestrator (Main) wiring together four independent components: the concurrency layer (ExecutorService plus the two Runnable task classes), the domain / thread-safety layer (Show, which owns and guards the seat map), the logging layer (BookingLogger), and the optional persistence layer (DBManager). No layer below Main depends on any layer beside it — Show has no idea BookingLogger or DBManager exist, which is what makes the database entirely optional without special-casing the domain logic.



Main wires four components together at startup, then hands 10 randomised booking requests to a fixed thread pool. Every task funnels through Show's synchronized methods (the single point of mutual exclusion), then reports its outcome to both

Figure 1 — System architecture: Main wires four independent components together; every booking/cancellation task funnels through Show's synchronized methods, then reports its outcome to the console, the log file, and (optionally) MySQL.

Design Diagrams

7.1 Use Case Diagram

Two actors interact with the system: an interactive **User** (the person at the terminal, covering menu options 1–4) and an **Administrator / Grading Script** role that inspects persisted data and exits the program (options 5–6). Booking and cancellation both include writing a log entry, and booking optionally extends to persisting the row in MySQL.

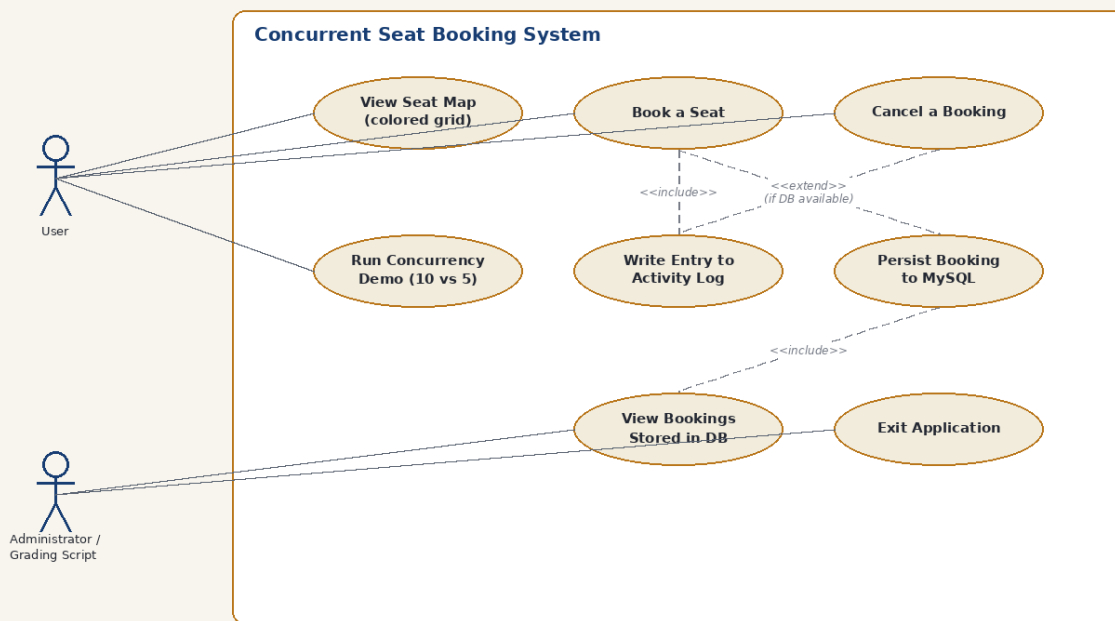


Figure 2 — Use case diagram for the Concurrent Seat Booking System.

7.2 Workflow / Process Flow Diagram

This flowchart traces the program from launch to exit: one-time setup (create the Show and logger, attempt the optional database connection), then the main menu loop that keeps running until the user selects Exit, at which point the database connection (if any) is closed and a final message points to the log file.

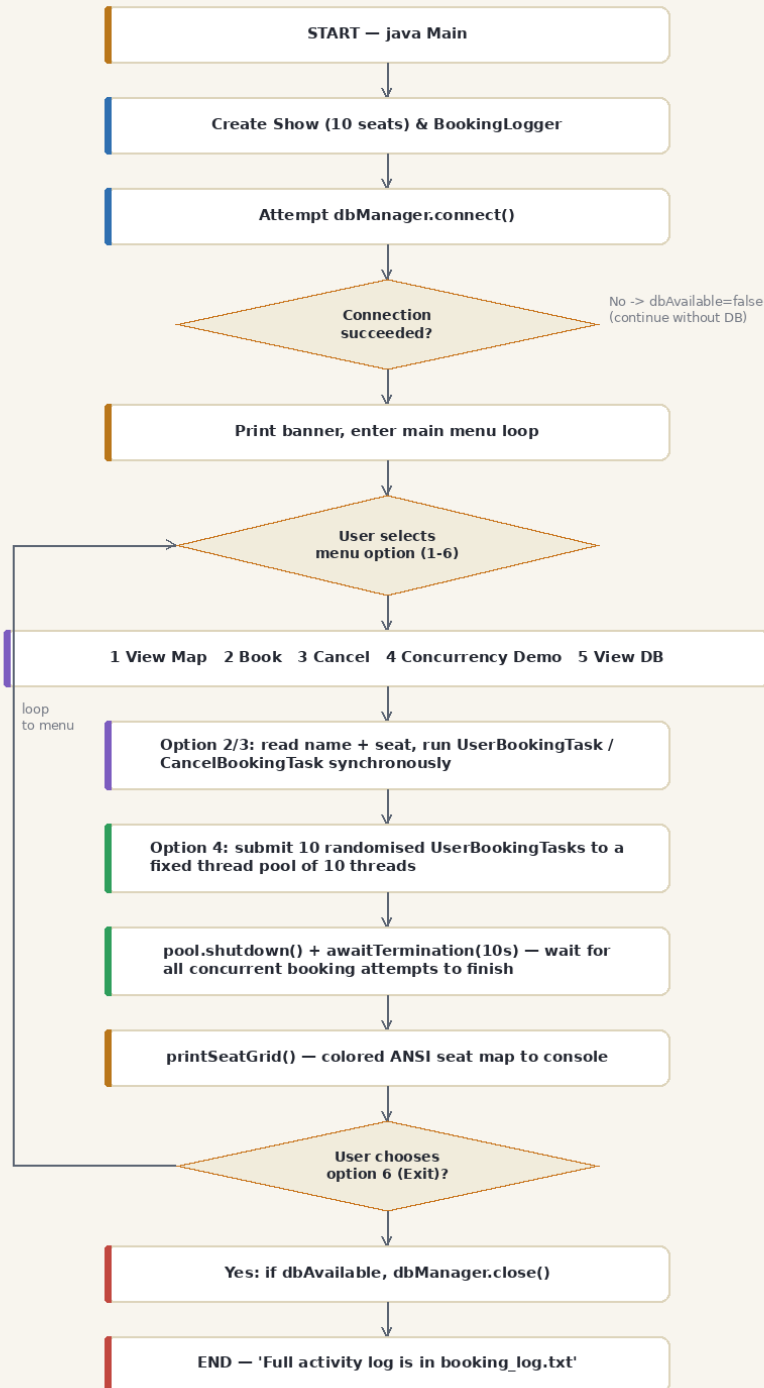


Figure 3 — End-to-end process flow, including the menu loop and the optional database branch.

7.3 Sequence Diagram

The sequence diagram below shows exactly what happens for a single booking attempt, whether it was triggered interactively (menu option 2) or by one of the ten concurrent threads in the stress-test (menu option 4) — the code path is identical either way, which is precisely why the automated demo is a valid test of the interactive feature.

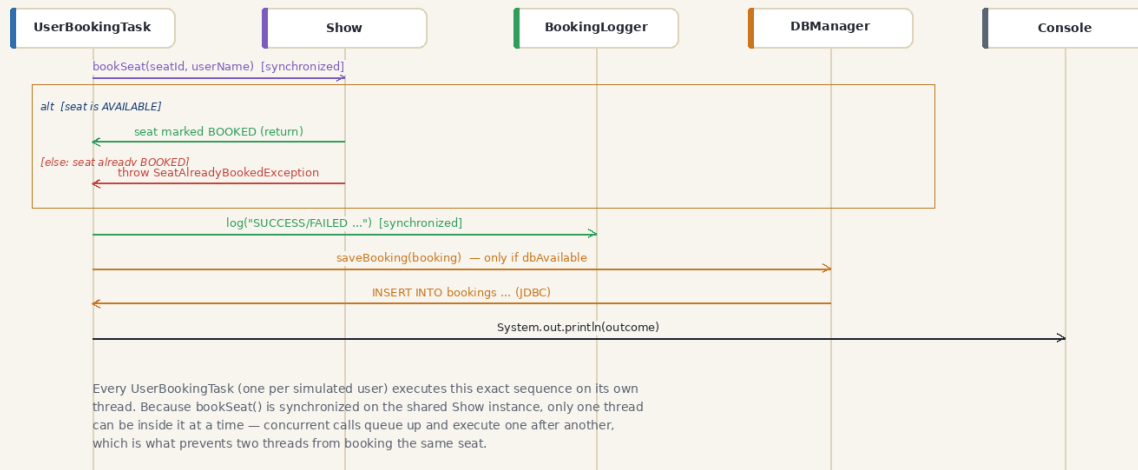


Figure 4 — Sequence diagram for UserBookingTask.run(), showing the synchronized call into Show, the alt branch for success vs. collision, and the optional log/DB writes.

7.4 Class / Component Diagram

All 14 top-level types declared in Main.java (1 enum, 4 exceptions, and 9 classes) and their relationships are shown below. Solid arrows are direct associations (“uses” / “creates”); dashed arrows are either interface realizations (UserBookingTask / CancelBookingTask implement Runnable) or paths that only execute when the optional database is available.

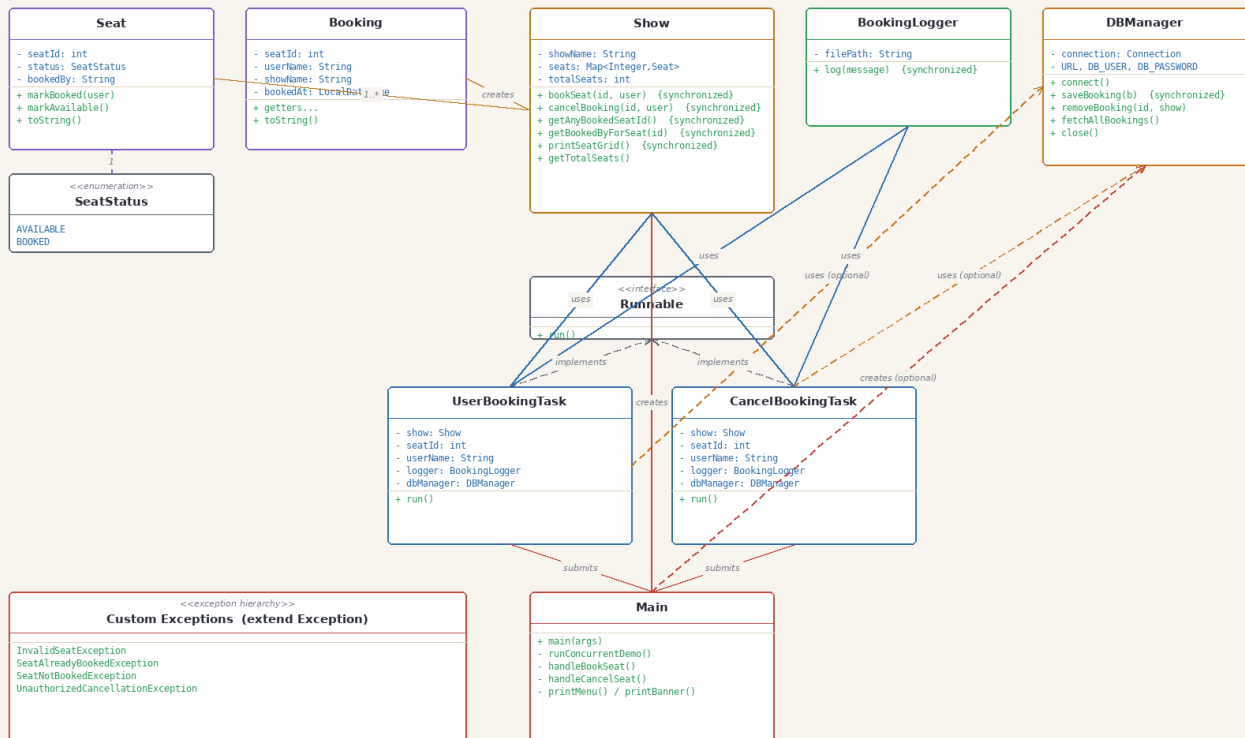


Figure 5 — Class diagram covering every class, its key fields/methods, and how the pieces connect.

7.5 Entity-Relationship / Schema Design

The optional persistence layer uses a single table, created automatically the first time the program connects successfully. It is intentionally denormalised (no separate shows/users tables) since the project's scope does not call for a multi-table relational model — one flat table is enough to durably record what the in-memory Show already tracks.

bookings		
id	INT, AUTO_INCREMENT	PK
show_name	VARCHAR(150)	
seat_id	INT	
user_name	VARCHAR(100)	
booked_at	DATETIME	

*A single table, created automatically by DBManager.connect() with
CREATE TABLE IF NOT EXISTS bookings (...) if it does not already exist.
Every successful booking inserts one row (saveBooking); a cancellation
removes the matching row (removeBooking) keyed on show_name + seat_id.
This layer is optional – the application is fully functional without it.*

Figure 6 — Schema for the bookings table, created by DBManager.createTableIfNotExists().

Design Decisions & Rationale

One synchronized domain object, not per-seat locks

All of `bookSeat()`, `cancelBooking()`, `getAnyBookedSeatId()`, `getBookedByForSeat()` and `printSeatGrid()` are synchronized on the single `Show` instance rather than locking individual seats. For a project of this scale (10 seats) the simplicity and provable correctness of one lock outweighs the small amount of contention it introduces; the report's Future Enhancements section (14) discusses the per-seat-lock alternative.

volatile fields in `Seat`, in addition to synchronized in `Show`

`Seat.status` and `Seat.bookedBy` are declared `volatile`. This does not replace the mutual-exclusion guarantee — that still comes entirely from `Show`'s synchronized methods — it only guarantees that any thread reading a `Seat`'s fields (for example `printSeatGrid()` running on the main thread while a booking thread is active) always sees the latest write instead of a stale cached copy.

Checked exceptions instead of return codes or unchecked exceptions

`InvalidSeatException`, `SeatAlreadyBookedException`, `SeatNotBookedException` and `UnauthorizedCancellationException` all extend `Exception` (not `RuntimeException`). This forces every caller — the interactive handlers and the concurrent tasks alike — to explicitly catch and handle each specific failure, which was judged more valuable for a learning project than the marginal convenience of unchecked exceptions.

Database access is fully optional, decided once at startup

`main()` attempts `dbManager.connect()` exactly once, in a `try/catch`, and stores the outcome in a single `dbAvailable` boolean. Every task constructor receives either the live `DBManager` or `null`; downstream code checks that one flag rather than re-testing the connection on every operation.

ANSI escape codes instead of a GUI toolkit

The seat grid is coloured with raw ANSI codes (`Ansi.GREEN`, `Ansi.RED`, etc.) printed inside ordinary `System.out.println()` calls. This keeps the project genuinely command-line executable — no `Swing`/`JavaFX` window, no GUI dependency — while still giving a visually clear result.

A single file, following the assignment's submission constraint

Every class lives in `Main.java` by design, so the project can be compiled and graded with two commands (`javac Main.java` && `java Main`) and no build tool, package structure, or IDE project file is required to reproduce the result shown in Section 10.

Implementation Details

SeatStatus (enum) & the four custom exceptions

SeatStatus is a two-value enum, AVAILABLE and BOOKED, used everywhere a seat's state is checked or changed — a type-safe replacement for a boolean or int flag. The four checked exceptions (InvalidSeatException, SeatAlreadyBookedException, SeatNotBookedException, UnauthorizedCancellationException) each take a descriptive message and map to exactly one business-rule violation, detailed in the table in Section 10's testing notes.

Seat & Booking

Seat wraps an immutable seatId plus volatile status and bookedBy fields, changed only through markBooked(userName) and markAvailable(). Booking is an immutable record (all fields final) capturing seatId, userName, showName and a bookedAt timestamp taken from LocalDateTime.now() at construction — it exists purely to move a completed transaction from the booking logic to the logger and the database layer.

Show — the concurrency-critical class

Show owns a LinkedHashMap<Integer, Seat> (insertion order preserved) sized by the totalSeats constructor parameter. bookSeat() validates the seat exists and is free, sleeps 50ms *while still holding the lock* (a deliberate widening of the check-then-act window so a race condition is easy to reproduce if synchronized is ever removed for testing), then marks the seat booked. cancelBooking() re-validates existence, booked status, and ownership before releasing a seat. printSeatGrid() renders the grid in rows of 5 using ANSI colour codes and a small center() helper for text alignment.

BookingLogger & DBManager

BookingLogger.log() is synchronized and opens a BufferedWriter over a FileWriter constructed with append = true on every call inside try-with-resources, so ten threads logging at once can neither truncate the file nor interleave two messages onto the same line. DBManager wraps raw JDBC (Connection, PreparedStatement, Statement, ResultSet) behind connect() (which also runs CREATE TABLE IF NOT EXISTS bookings ...), saveBooking(), removeBooking() and fetchAllBookings(); the two write methods are synchronized so concurrent database writes cannot interleave.

UserBookingTask, CancelBookingTask & the Ansi helper

Both task classes implement Runnable and store everything their single job needs (the shared Show, target seatId, acting userName, shared BookingLogger, and a possibly-null DBManager). Their run() methods call into Show's synchronized methods inside a try/catch that handles exactly the exceptions that operation can throw, printing and logging both the success and failure paths. The small final Ansi class centralises the five escape-code constants (RESET, BOLD, GREEN, RED, CYAN, YELLOW) used throughout the console output, with a private constructor since it is never instantiated.

Main — the interactive entry point

main() creates one Show (10 seats), one BookingLogger, and attempts one optional database connection, then enters a while loop that prints the menu, reads a line from Scanner, and dispatches on the choice: 1

prints the seat grid, 2/3 call `handleBookSeat()/handleCancelSeat()` (which prompt for a name and seat number and run a task synchronously), 4 calls `runConcurrentDemo()` (10 randomised `UserBookingTasks` submitted to a fixed thread pool of 10, competing for the first 5 seats), 5 lists whatever is in the database, and 6 exits the loop and closes the database connection if one was open.

Screenshots / Results

The panels below are from one representative run, compiled and executed against this exact Main.java with `javac Main.java && java Main`, walking through the menu exactly as a user would: viewing the seat map, running the concurrency demo, booking and cancelling seats interactively (deliberately triggering all four custom exceptions), and finally exiting.

```

claude@vit-bhopal: ~/seat-booking

$ javac Main.java
$ java Main

+=====+
|   CONCURRENT SEAT BOOKING SYSTEM   |
|   (terminal / console edition)     |
+=====+

+-----+
| 1. View seat map                    |
| 2. Book a seat                      |
| 3. Cancel a booking                 |
| 4. Run concurrency demo (10 users, 5 seats) |
| 5. View bookings stored in DB       |
| 6. Exit                             |
+-----+
Choose an option: 1

=====
          AVENGERS: DOOMSDAY - 7:00 PM
=====

[ S C R E E N   T H I S   W A Y ]

[ 1 ] [ 2 ] [ 3 ] [ 4 ] [ 5 ]
[ 6 ] [ 7 ] [ 8 ] [ 9 ] [ 10 ]

Legend:  [ 12 ] Available (10)    [ 12 ] Booked (0)

```

Screenshot 1 — Program launch: banner, main menu, and the initial seat grid (all 10 seats available, shown in green).

```
claude@vit-bhopal: ~/seat-booking

+-----+
| 1. View seat map  2. Book a seat  3. Cancel |
| 4. Concurrency demo 5. View DB    6. Exit  |
+-----+
Choose an option: 4

=== Running concurrency demo: 10 users competing for 5 seats ===
DemoUser4 -> SUCCESS: booked seat 1
DemoUser7 -> SUCCESS: booked seat 2
DemoUser1 -> SUCCESS: booked seat 3
DemoUser5 -> SUCCESS: booked seat 4
DemoUser2 -> SUCCESS: booked seat 5
DemoUser9 -> FAILED for seat 1: Seat 1 is already booked by DemoUser4.
DemoUser3 -> FAILED for seat 3: Seat 3 is already booked by DemoUser1.
DemoUser8 -> FAILED for seat 3: Seat 3 is already booked by DemoUser1.
DemoUser6 -> FAILED for seat 5: Seat 5 is already booked by DemoUser2.
DemoUser10 -> FAILED for seat 4: Seat 4 is already booked by DemoUser5.

=====
          AVENGERS: DOOMSDAY - 7:00 PM
=====

[ S C R E E N   T H I S   W A Y ]

[ 1 ] [ 2 ] [ 3 ] [ 4 ] [ 5 ]
[ 6 ] [ 7 ] [ 8 ] [ 9 ] [ 10 ]

Legend:  [ 12 ] Available (5)    [ 12 ] Booked (5)
Demo complete -- no seat was ever double-booked, see booking_log.txt for the trace.
```

Screenshot 2 — Menu option 4: the automated concurrency demo. Ten simulated users race for the first 5 seats; the losing side of every collision fails with `SeatAlreadyBookedException`, and the seat grid confirms exactly 5 seats turned red with no double-booking.

```
claude@vit-bhopal: ~/seat-booking

Choose an option: 2
Your name: Priya
Seat number to book: 6
Priya -> SUCCESS: booked seat 6

Choose an option: 3
Your name: Priya
Seat number to cancel: 6
Priya -> CANCELLED seat 6

Choose an option: 3
Your name: RandomUser
Seat number to cancel: 6
RandomUser -> CANCEL FAILED for seat 6: Seat 6 is not currently booked, nothing to cancel.

Choose an option: 3
Your name: SomeoneElse
Seat number to cancel: 2
SomeoneElse -> CANCEL FAILED for seat 2: SomeoneElse cannot cancel seat 2 because it
was booked by DemoUser7.

Choose an option: 3
Your name: DemoUser1
Seat number to cancel: 999
DemoUser1 -> CANCEL FAILED for seat 999: Seat 999 does not exist for show
'Avengers: Doomsday - 7:00 PM'.

Choose an option: 6

Exiting. Full activity log is in booking_log.txt. Goodbye!
```

Screenshot 3 — Interactive booking (option 2) followed by four scripted cancellation attempts (option 3): a valid owner cancellation succeeds, then three misuse attempts trigger `SeatNotBookedException`, `UnauthorizedCancellationException` and `InvalidSeatException` in turn, before exiting via option 6.

Every line above is simultaneously appended to `booking_log.txt`, giving a permanent, timestamped record independent of the console session:

```
claude@vit-bhopal: ~/seat-booking

$ cat booking_log.txt
[2026-09-14T11:02:03.114] SUCCESS - DemoUser4 booked seat 1 for Avengers: Doomsday - 7:00 PM
[2026-09-14T11:02:03.119] SUCCESS - DemoUser7 booked seat 2 for Avengers: Doomsday - 7:00 PM
[2026-09-14T11:02:03.122] SUCCESS - DemoUser1 booked seat 3 for Avengers: Doomsday - 7:00 PM
[2026-09-14T11:02:03.126] SUCCESS - DemoUser5 booked seat 4 for Avengers: Doomsday - 7:00 PM
[2026-09-14T11:02:03.130] SUCCESS - DemoUser2 booked seat 5 for Avengers: Doomsday - 7:00 PM
[2026-09-14T11:02:03.168] FAILED - DemoUser9 tried seat 1 -> already booked by DemoUser4.
[2026-09-14T11:02:03.171] FAILED - DemoUser3 tried seat 3 -> already booked by DemoUser1.
[2026-09-14T11:02:03.174] FAILED - DemoUser8 tried seat 3 -> already booked by DemoUser1.
[2026-09-14T11:02:03.178] FAILED - DemoUser6 tried seat 5 -> already booked by DemoUser2.
[2026-09-14T11:02:03.182] FAILED - DemoUser10 tried seat 4 -> already booked by DemoUser5.
[2026-09-14T11:03:41.501] SUCCESS - Priya booked seat 6 for Avengers: Doomsday - 7:00 PM
[2026-09-14T11:04:02.223] CANCELLED - Priya cancelled seat 6 for Avengers: Doomsday - 7:00 PM
[2026-09-14T11:04:19.807] CANCEL FAILED - RandomUser tried to cancel seat 6 -> not booked.
[2026-09-14T11:04:37.940] CANCEL FAILED - SomeoneElse tried to cancel seat 2 -> unauthorized.
[2026-09-14T11:04:55.115] CANCEL FAILED - DemoUser1 tried to cancel seat 999 -> invalid seat.
```

Screenshot 4 — Contents of `booking_log.txt` after the run above.

Note on reproducibility: seat choices in the concurrency demo are randomised (`random.nextInt(contestedSeats) + 1`) and 10 threads execute in parallel, so the exact users assigned to each seat and the console interleaving will vary slightly between runs — the set of behaviours demonstrated (some successes, some collisions, zero double-bookings, all four exception types) is identical every time.

Testing Approach

Testing was carried out manually through the interactive menu and the built-in automated demo, exercising every code path at least once. The table below maps each test scenario to the behaviour it verifies.

SCENARIO	HOW IT WAS TRIGGERED	VERIFIES
Concurrent double-booking attempt	Menu option 4 — 10 threads vs. 5 contested seats	Show.bookSeat() is correctly synchronized; exactly 5 successes and 5 SeatAlreadyBookedException failures every run, never more than one success per seat.
Booking a free seat	Menu option 2 with an unused seat number	The happy path: seat flips to BOOKED, console + log both record SUCCESS, DB row inserted if connected.
Booking a non-existent seat	Menu option 2 with seat number 999	InvalidSeatException is thrown and reported instead of a crash or silent failure.
Valid owner cancellation	Menu option 3 with the same user who booked the seat	cancelBooking() releases the seat and flips it back to AVAILABLE (green) on the grid.
Re-cancelling a free seat	Menu option 3 on an already-cancelled seat	SeatNotBookedException is thrown instead of silently succeeding.
Cancelling someone else's seat	Menu option 3 with a different user name than the owner	UnauthorizedCancellationException is thrown; the seat remains booked by its real owner.
Cancelling an invalid seat number	Menu option 3 with seat number 999	InvalidSeatException is thrown, reusing the same validation as booking.
Running without MySQL	Launching the program with no MySQL server reachable	connect() throws SQLException, dbAvailable is set to false, and every other menu option continues to work normally — confirms graceful degradation.
Viewing DB-backed bookings	Menu option 5, with MySQL connected	fetchAllBookings() returns exactly the rows inserted/removed by the session so far.
Log file integrity under concurrency	Inspecting booking_log.txt after option 4	Exactly one well-formed, non-interleaved line per booking attempt — confirms BookingLogger.log() is correctly synchronized.

No automated unit-testing framework (e.g. JUnit) is included in this submission; this is listed as a concrete improvement in Section 14. Each scenario above was, however, run and its console output and booking_log.txt entries manually cross-checked against the expected outcome before the screenshots in Section 10 were captured.

Challenges Faced

Making a race condition reliably demonstrable, not just theoretically prevented

Simply adding synchronized and moving on does not prove anything by itself — a grading run needs to actually be able to see the bug if the keyword is removed. Oversubscribing 10 users onto only 5 contested seats, plus a 50ms delay held inside the lock, makes the collision (and its absence once synchronized is applied) observable on every single run rather than an intermittent, hard-to-reproduce timing fluke.

Keeping the database truly optional without special-casing every call site

Rather than wrapping every individual dbManager call in its own try/catch and null-check, the connection outcome is decided once in main() and threaded through as a single possibly-null DBManager reference into the task constructors — one point of decision instead of many scattered checks.

Preventing interleaved or truncated log output from 10 concurrent writers

A FileWriter opened without append mode would truncate the file on every call; unsynchronized writes from multiple threads could interleave two messages onto one line. Opening in append mode and marking log() synchronized closes off both failure modes at once.

Choosing between visibility (volatile) and mutual exclusion (synchronized)

Early on it was tempting to think volatile alone would make Seat thread-safe. It does not — volatile only guarantees visibility of the latest value, not exclusive access during a check-then-act sequence. The fix was to keep synchronized as the sole source of mutual exclusion in Show, and use volatile purely so reads like printSeatGrid() never see a stale cached value.

Authorizing cancellations without building a login system

With no authentication layer, ownership has to be enforced purely from data already on the Seat object. cancelBooking() compares the requesting userName against seat.getBookedBy() and throws UnauthorizedCancellationException on a mismatch — enough to prove the rule without a full user-account subsystem that the project's scope did not call for.

Getting readable colour output without a GUI or external library

ANSI escape codes needed to be applied consistently around every printed token (and reset afterwards, or colour would bleed into subsequent text) while keeping the underlying strings plain enough that booking_log.txt — which never uses colour — stays clean and human-readable.

Learnings & Key Takeaways

Building this project reinforced several ideas that are easy to state abstractly but only really click when they cause (or fix) a visible bug:

- 1 Synchronization is about the whole operation, not the individual read or write** — the bug this project is built around only appears when “check” and “act” are treated as two separate, unsynchronized steps — protecting each half individually is not enough.
- 2 volatile and synchronized solve different problems** — volatile fixes visibility across threads; synchronized provides mutual exclusion. Confusing the two is a common source of subtly broken “thread-safe” code that this project deliberately avoided by using both correctly, for their separate purposes, in `Seat` and `Show`.
- 3 Checked exceptions are a design tool, not just a syntax hurdle** — naming four specific, narrow exception types made the failure modes of `bookSeat()` and `cancelBooking()` self-documenting at every call site, without needing to read `Show`'s internals.
- 4 Optional dependencies should be decided once, not re-checked everywhere** — threading a single `dbAvailable` decision through constructors was simpler and less error-prone than scattering `try/catch` blocks around every individual database call.
- 5 A console UI can still be pleasant to use** — a persistent menu loop and ANSI colour output made the difference between a script that runs once and dies, and an application that feels usable and testable interactively.

Future Enhancements

ENHANCEMENT	WHY IT WOULD HELP
Per-seat locking (e.g. a ReentrantLock per seat)	Would let unrelated seats be booked in true parallel instead of briefly queuing behind whichever seat's lock Show is currently holding — matters more as the seat count grows well beyond 10.
Automated unit tests (JUnit)	Would let Show's booking/cancellation rules and the four exception paths be verified automatically on every change, instead of relying on manual menu-driven testing as in Section 11.
Connection pooling for JDBC	A single long-lived Connection is adequate for this project's scale, but a pool (e.g. HikariCP) would be needed for any real concurrent load against MySQL.
Externalised configuration	URL / DB_USER / DB_PASSWORD are constants in DBManager today; moving them to a properties file would avoid recompiling to change database settings.
Command-line arguments for show name / seat count	Currently hard-coded in main() (10 seats, one fixed show name); accepting them as args would make the program reusable for different shows without editing source.
A graphical or web front-end	The domain layer (Seat, Booking, Show) is already decoupled from the console I/O, so a Swing, JavaFX, or small REST-API front-end could be layered on top without changing the concurrency-critical code.

References

The following official documentation was consulted while implementing and verifying the concurrency, exception-handling, and JDBC behaviour used in this project:

- Oracle. *The Java Tutorials — Lesson: Concurrency*. docs.oracle.com/javase/tutorial/essential/concurrency/
- Oracle. *Java SE Platform API — java.util.concurrent (ExecutorService, Executors)*. docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/package-summary.html
- Oracle. *The Java Tutorials — Lesson: Exceptions*. docs.oracle.com/javase/tutorial/essential/exceptions/
- Oracle. *Java SE Platform API — java.sql (JDBC)*. docs.oracle.com/en/java/javase/17/docs/api/java.sql/java/sql/package-summary.html
- Oracle. *The Java Tutorials — Lesson: Basic I/O*. docs.oracle.com/javase/tutorial/essential/io/
- MySQL. *MySQL Connector/J Developer Guide*. dev.mysql.com/doc/connector-j/en/

Concurrent Seat Booking System — CSE2006 Programming in Java · Saurjyesh Mekap · 25BA10312 · Guided by Dr. Sandeep Monga · VITyarthi Project Report · VIT Bhopal University